

Indoor navigation among walking people: a measured pipeline in Isaac Sim

Dimitrios Gkiokas · dimitris.gkiokas@outlook.com · July 2026, revised October 2026

Abstract. I built a simulation benchmark that takes one robot from reactive wandering to goal-directed autonomy in stages, and measures each stage against simulator ground truth. Worlds are generated from seeds: four rooms, doors of controlled width, furniture, and three people who stand still or walk scripted routes. Door-crossing success falls from 12 of 14 approached doors above 0.80 m to 2 of 7 below the robot's 0.66 m nominal minimum, on small counts. In 75 runs, walking people changed neither the median exploration speed nor the deadlock count, but the scripted walkers shoved the robot in up to half of the runs. The robot built all 25 maps autonomously among walking people (median 63% wall coverage in 20 wall-clock minutes). On those maps an offline reachability check predicted 85 of 100 goals feasible, and the live Nav2 stack truly reached 11, with 6 more arrivals reported that never happened. Retuning the localizer removed the false arrivals in the tours that could be checked and changed the true count by one. The main results are the navigator, which went from a terminal deadlock in all 25 v1 runs to none in 75 runs of v2.9.1, and the measurement method, whose ground-truth checks changed the conclusion at every stage, including two flaws in my own scenario. All code, data and analysis are public.

1. Introduction

The question behind this project is practical: what does it take for a small wheeled robot to move with purpose through cluttered rooms while people walk around it? Rather than answer with one demonstration, I broke the problem into stages that each produce a number: how narrow a door the reactive layer can pass, what human traffic costs, how good a map the robot can build for itself under that traffic, and how far a standard navigation stack gets on that self-built map. The stages share one discipline. Every result is checked against the simulator's ground truth, never against the robot's own belief. That choice carries the main finding of the last experiment, where the stack's self-reported successes and its actual positions disagree by metres, and it also exposed a problem in my own scenario, described in Section 4.

The pipeline runs in NVIDIA Isaac Sim 6.0 on Windows with ROS 2 Jazzy in WSL2, on one RTX 2080 Ti, below the simulator's minimum specification. Real-time factors sat between 0.35 and 0.55 for the whole campaign. I mention the hardware because the engineering that keeps a pipeline split across two operating systems and two clocks consistent at half speed is part of the work: wall-clock stamps everywhere, file-based handshakes between the simulator and the ROS side, and batch machinery that ran 181 batch runs unattended.

2. The benchmark factory

A world is a function of a seed. The generator splits a 20×20 m arena into four rooms with two crossing partition walls, cuts one door per half-wall with widths drawn from two bands (two anchor doors at 0.72–0.95 m, the other two at 0.50–0.78 m), scatters furniture, and writes a manifest CSV: every wall, door and obstacle with coordinates, plus walking routes for the three people. The manifest is the ground truth that every later measurement joins against. People walk their routes at constant speed with per-waypoint pauses and do not react to the robot. In the last walking condition one person per world is routed through a door to a neighbouring room (the widest one with a clear route), which turns that doorway into a contested resource. The camera sees the people's character meshes, the lidar a 0.25 m cylinder that moves with each person.

The robot is a 0.42 m differential-drive body. Its lidar is a 360-ray PhysX raycast published to /scan at about 20 Hz. Odometry integrates ground-truth motion with multiplicative noise (3% linear, 5% angular), which produced measured drifts up to 1.57 m in the mapping runs. A logger records robot and people ground-truth positions at 10 Hz into one CSV per run. Batches run unattended: the simulator side regenerates worlds and signals the WSL side through files. A deadlock watchdog ends a run when its whole position trail stays inside a 0.5 m box for 180 wall-clock seconds. An earlier watchdog compared only the trail's endpoints and killed 16 of 25 healthy runs whose paths looped back near old positions. The post-mortem of that batch is what fixed the rule.

3. Reactive navigation vs door width

The reactive layer is a follow-the-gap navigator over the raw scan with a stack of recovery behaviours, each added after a failure observed in an earlier version, plus a camera-based person brake (YOLOv8n). Its predecessor, the same core without the recovery layers, reached a terminal deadlock in all 25 worlds (median onset 24 s). Version 2.9.1 completed 75 runs of 20 wall-clock minutes, static and with walking people, without a terminal deadlock. At doors, success depends on width. Of the doors it approached, the robot crossed 12 of 14 between 0.80 and 0.95 m, 10 of 17 between 0.66 and 0.80 m, and 2 of 7 below 0.66 m, the nominal minimum (0.42 m body plus 0.12 m inflation on each side). Only 38 of the 100 doors were approached, so the curve shows a trend rather than a sharp threshold (Figure 1). The generator samples the harder doors around that boundary on purpose, so most worlds have a door near it.

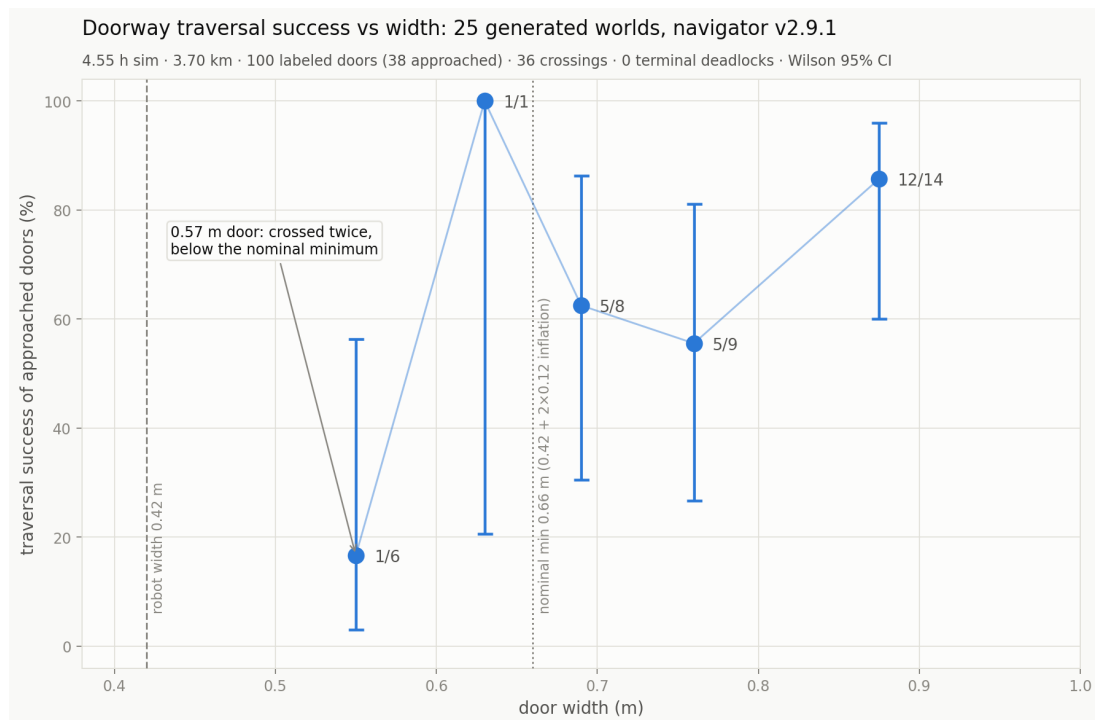


Figure 1. Door traversal success against door width, 25 seeded worlds, Wilson 95% intervals. Dashed line: the 0.42 m robot width. Dotted line: the 0.66 m nominal minimum.

4. What walking people cost

The same 25 worlds ran under three conditions: the three people standing still, 1–3 of them walking routes inside their rooms, and the same with one person per world crossing rooms through a contested door. That is 75 runs of 20 wall-clock minutes, with identical geometry per seed because the generator draws all routes after placement. Median exploration speed did not move: 13.8, 13.8 and 13.9 m/min. No run ended in a terminal deadlock. Door crossings rose from 7.9 to about 10 per simulated hour, and encounters in the zone of the contested door rose from 26 to 54, while the robot kept using that door (17 crossings with and without the cross-room walker).

The ground-truth logs also show something I first missed. The walkers follow their routes without yielding, and they walk into the robot. The robot is about 0.4 m wide and each person's collision cylinder has a 0.25 m radius, so their shapes meet at roughly 0.40–0.45 m centre to centre, and most walking runs had a closest pass below 0.45 m. A hard push appears in the log as the robot moving faster than 1 m/s, against a top speed of 0.4 m/s, with a person within 0.5 m. In 39 of the 41 such events across all walking batches the robot was displaced away from the person. Hard pushes happened in 7 of the 25 same-room runs and 12 of the 25 cross-room runs, mostly from the side or from behind, and never with the people standing still. These counts are a lower bound, since lighter contacts do not move the robot that fast. The closest pass of the 75 runs, 0.27 m centre to centre, was one of them: the cross-room walker came up behind the robot at a 0.67 m door and pushed it 3 m. The person brake looks only at the middle third of the forward camera image, so it could not react. My earlier near-contact threshold of 0.35 m sits below the distance at which the two shapes meet, which is why it flagged only one of these events. The contacts say as much

about the scripted walkers as about the navigator, but a navigator whose person brake only looks forward has no way to avoid them.

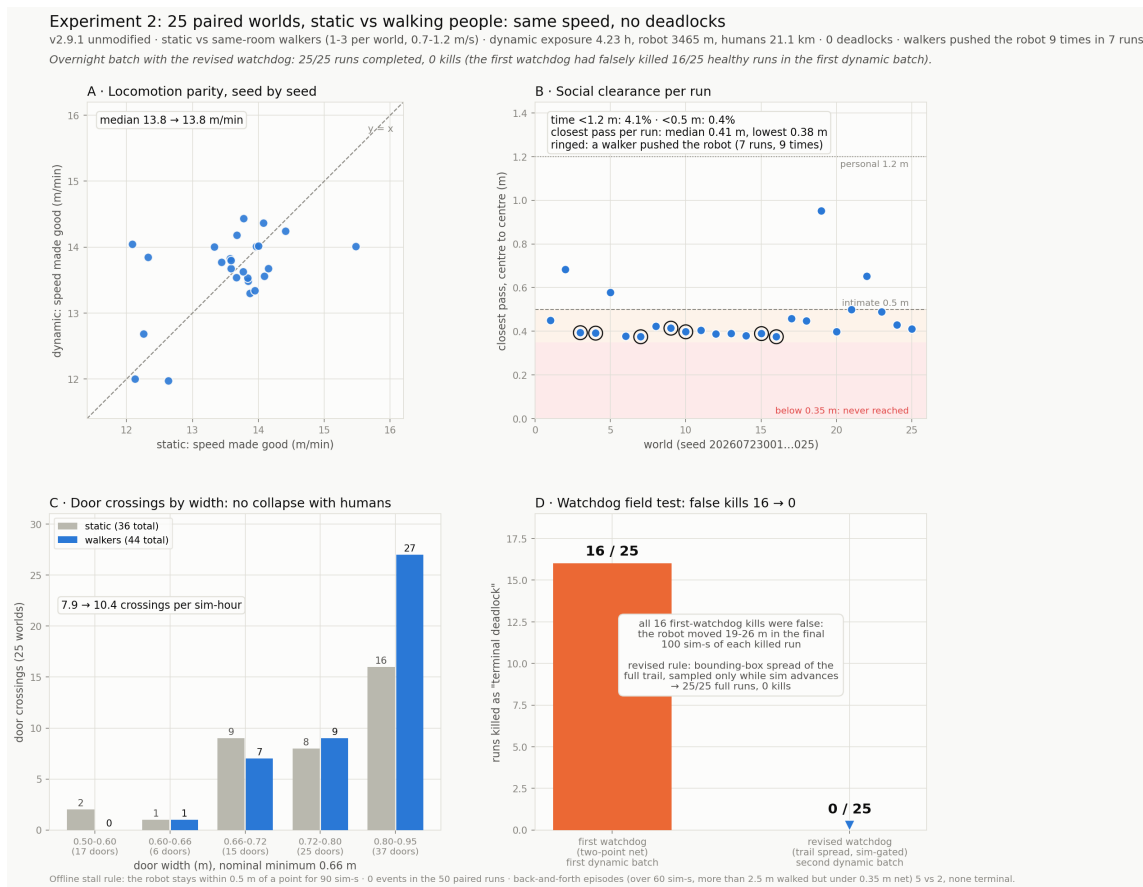


Figure 2. Static vs walking people across 25 worlds: speed seed by seed, closest pass per run (ringed where a walker pushed the robot), door crossings by width, and the watchdog field test.

5. Mapping while people walk

Stage three hands the same wandering robot to slam_toolbox: 25 worlds, 20 wall-clock minutes each (about 10 simulated minutes), odometry noise and walkers active, one map saved per world by an orchestrator that restarts SLAM between worlds. All 25 maps were saved without intervention in one unattended session. Two results matter later. First, I found no sign that walking people degrade the maps: time spent within 1.2 m of a person is barely correlated with coverage ($r = -0.13$), and in a single-world test the walking people left almost no trace in the map, while the same people frozen in place were mapped as obstacles. Second, the wander policy is the bottleneck: median wall-surface coverage is 63% (range 47–92%), against 82% median occupied-cell precision. The robot maps what it visits accurately. It just does not visit enough. Doors come out narrower on the map than in truth by a median of 5 cm, which matters at a planner's inflation radius. Measured odometry drift correlates mildly with map precision ($r = -0.37$, $n = 25$).

Experiment 3: twenty-five worlds mapped in one unattended session

v2.9.1 wanders 20 wall-clock min per generated world (walking people, odometry noise 3%/5%) · slam_toolbox restarted per world by the WSL orchestrator · 25/25 maps saved autonomously

grey = mapped occupancy · red = manifest ground truth · label: surface coverage

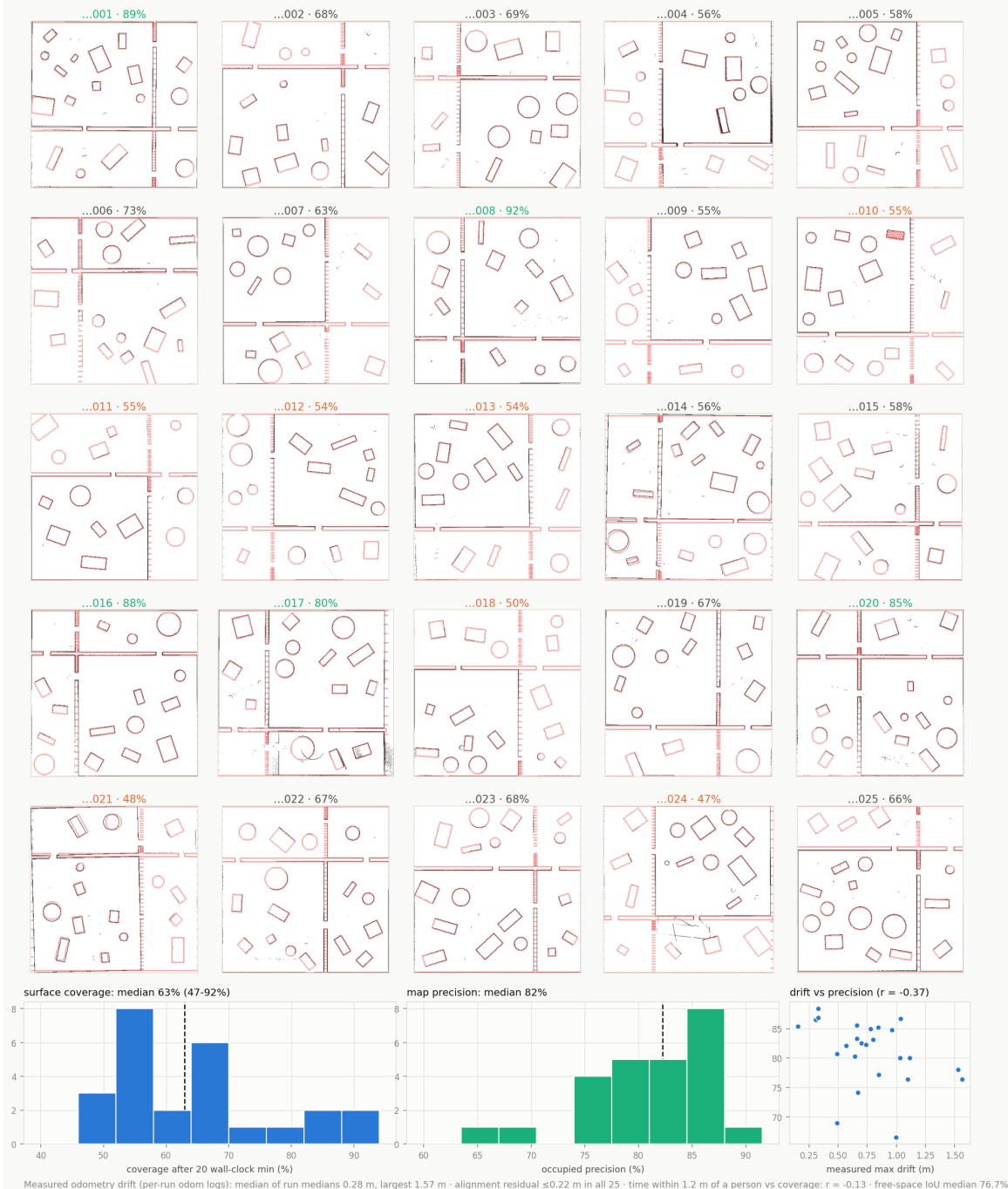


Figure 3. Twenty-five worlds mapped in one unattended session. Grey: mapped occupancy. Red: manifest ground truth. Per-world label: wall-surface coverage.

6. Goal-directed missions on self-built maps

The final stage gives Nav2 (AMCL, NavFn, DWB) each world's self-built map and a four-goal tour: the centres of the three other rooms, then home. Before running anything I computed, per world, which goals are reachable at the planner's 0.30 m radius on the map and on the ground-truth geometry (a BFS over the inflated grid). The check says 85 of 100 goals are feasible on the maps, and flags four worlds as cages whose spawn room has no door the conservative planner will accept.

The live runs then tested that prediction. With baseline AMCL settings (v8) the stack truly reached 11 of 100 goals, where truly means the ground-truth position at the moment of reported success lies within 1.5 m of the goal: 7 of the 60 cross-room goals the check calls feasible, plus 4 returns home, three of them in worlds the robot never left. Six

more goals were reported reached and were not, in four worlds. The worst case reported the first two goals reached after 0.1 s each while the robot was still at its spawn point, 7.6 and 6.8 m from them. In 7 worlds the robot did reach an adjacent room, 0.12–0.93 m from the goal. The four predicted cage worlds refused every cross-room goal, as they should.

I then retuned AMCL for sparse maps (v9: 500–4000 particles instead of 300–1500, a wider motion model, more tolerance for beams that the map cannot explain) and re-ran all 25 tours: a paired experiment with one changed variable. In the tours that could be checked, the tuning removed every false arrival (6 to 0) and every world with false arrivals (4 to 0). The true count moved from 11 to 12, and real cross-room arrivals from 7 of 75 to 8 of 72 checkable. The failures changed form: worlds that previously reported false arrivals now time out or stay wedged (wedged worlds 4 to 8). The cage worlds refused again, in both configurations, 8 of 8 tours as predicted. One v9 tour (world 007) has no matching ground-truth log and is excluded, but it reported all four goals reached within 4.3 s, which cannot be real, so the tuning did not remove false arrivals entirely. My hypothesis is that the maps set the ceiling: a better estimator stops reporting arrivals that did not happen, but it cannot recover walls the 20-minute map never captured. Map coverage alone correlates only weakly with real arrivals ($r = 0.26$ and 0.36), so this remains a hypothesis (Figure 4).

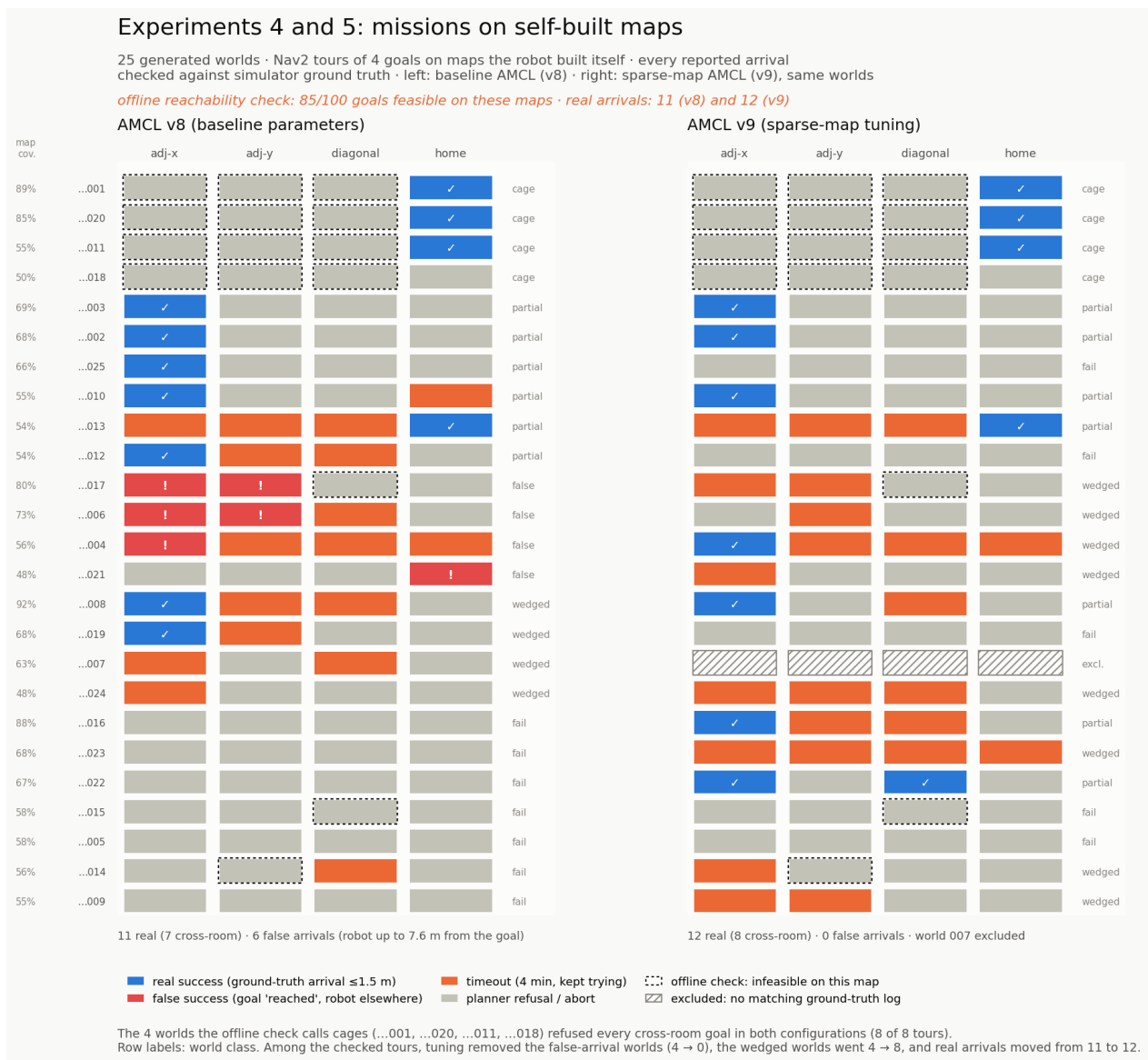


Figure 4. The paired mission experiment: 25 worlds × 4 goals, baseline AMCL (v8, left) vs sparse-map tuning (v9, right), every cell checked against ground truth. Dotted cells: goals the offline check calls infeasible on that map. Hatched: excluded, no matching ground-truth log.

7. Main conclusion

The project has two main results. The first is the navigator. The same follow-the-gap core, with recovery layers added one observed failure at a time, went from a terminal deadlock in all 25 worlds (v1, median onset 24 s) to none (v2.9.1). In the same worlds it travelled 3.7 km instead of 245 m and crossed doors 36 times instead of 3, once through a 0.57 m door below its nominal minimum width. It kept that behaviour among walking people: 75 runs across the three conditions without a terminal deadlock, at an unchanged median speed of about 13.8 m/min, and it explored all 25 worlds of the mapping batch unattended. Its two limits are measured as well. Its person brake only looks forward, so walkers that do not yield can push it, and as an explorer it covers a median 63% of the walls in 20 minutes.

The second result is the way all of this was measured. Checking every result against the simulator's ground truth changed the conclusion at each stage. It showed that the mission stack reported 6 arrivals that never happened, two of them while the robot was still at its spawn point, that the first deadlock watchdog had killed 16 healthy runs, and that one mapping run had silently used zero odometry noise. It also found flaws in my own scenario: the scripted walkers pushed the robot in up to half of the walking runs, while my near-contact threshold flagged only one event. None of these appeared in the stack's self-reported results or in the batch runner's verdicts. On the self-built maps, tuning the localizer removed the false arrivals without raising the real ones above 12 of 100, which points to map coverage, and so to the explorer, as the next limit to test.

8. What I would tell the next builder

First, verify against ground truth from day one, and check the scenario as hard as the robot. Second, on a pipeline split between Windows and WSL, prefer explicit signal files to clocks and file attributes. WSL's attribute cache served stale timestamps and broke a trigger that file existence handled cleanly. Third, long-running ROS 2 batches need hygiene between iterations: shared-memory segments from killed DDS participants accumulate until the command-line tools stop seeing the graph. Fourth, inside the simulator, cache no handles across stage reloads. Rebinding on every play is what let one session survive many world regenerations.

9. Limits

Twenty-five worlds per condition are enough for the patterns reported here, not for fine statistics. The feasibility check is a BFS approximation of the planner, not the planner. The 1.5 m arrival threshold absorbs the timing tolerance of the offline log join. The walkers do not react to the robot, and hard contacts are detected from the logged robot motion, since the simulator's collision events were not logged, so lighter contacts are not counted. Robot to person distances are centre to centre. Everything ran in one simulator on one machine, and the wall-clock timing design, which is why the stack tolerates real-time factor swings, does not emulate a real robot's clock discipline.

10. Future work

The mapping stage points at coverage as the bottleneck. Frontier-based exploration in place of reactive wander would test whether better maps raise the ceiling that localizer tuning could not move. On the social side, walkers that react to the robot and a navigator that watches its rear with the 360-degree lidar would address the contacts of Section 4. Smaller follow-ups: wheel-encoder odometry in place of the noise model, and a backtrack recovery that retreats along the entry path instead of spinning in place.